ASSESSMENT REPORT

Mysoftheaven (BD) Ltd.

A Complete IT Solution

MYSOFT HEAVEN (BD) LTD.

PREPARED BY SUDEEP MONDAL DEEP

Assessment 1

1. Application Flow

The system begins when a user submits a lead message through either the web form at the root URL or directly via the POST /webhook/lead endpoint. The FastAPI server receives the message and immediately runs input validation empty messages and anything over 5,000 characters are rejected before any further processing occurs.

Once the message passes validation, it is passed to the LLM handler which calls the Groq API (LLaMA 3.3 70B model). The raw message is sent alongside a structured system prompt that instructs the model to perform three tasks in a single API call: classify the intent, extract key fields, and draft a response. The model returns a single JSON object containing all three outputs at once.

The parsed result is then written to a SQLite database with a timestamp. Whether the LLM call succeeded or the fallback was used, the lead is always saved. Finally, the structured result is returned to the caller as a JSON response, and the web UI updates to display the intent badge, extracted fields, and the AI-generated reply.

2. AI Logic and Automation

The core AI logic is contained in a single system prompt divided into three explicit tasks. This single-prompt, multi-task design means only one API call is needed per lead, which reduces latency and cost.

Intent Classification — The model is given three strictly defined categories: Sales (buying interest, demo requests, pricing inquiries), Support (existing customer issues, technical problems), and Spam (irrelevant, promotional, or nonsensical messages). The definitions are written to be mutually exclusive so the model has clear decision boundaries.

Field Extraction — The model is instructed to extract name, company name, and requirement from the message. Critically, it is told to return null for any field it cannot find with confidence. This explicit instruction prevents the model from hallucinating plausible-sounding but incorrect values.

Response Generation — The model writes a short, personalized reply of three to five sentences. The length cap is enforced via the max_tokens limit at the API level, not just through prompt instruction, so it cannot be bypassed by the model.

After the API response is received, the server validates the intent value against the three allowed strings. If the model returns anything outside of Sales, Support, or Spam — which can happen with borderline messages — the value is corrected to Support server-side before saving.

3. Key Design Decisions

Groq with LLaMA 3.3 70B instead of OpenAI — Groq's inference API is OpenAI-compatible, meaning the integration required only a URL and model name change. LLaMA 3.3 70B performs comparably to GPT-3.5 for structured extraction tasks and is available on Groq's free tier, making the project accessible without billing setup.

Single-prompt multi-task design — Classification, extraction, and response generation are handled in one prompt rather than three sequential calls. This reduces API round trips from three to one, cuts latency significantly, and simplifies error handling since there is only one point of failure per lead.

SQLite as the datastore — SQLite requires zero configuration, runs as an embedded file, and is sufficient for the workload of a lead intake system. It was chosen over PostgreSQL or MongoDB to keep the project self-contained and runnable without any external services.

Fallback always saves the lead — If the LLM is unavailable for any reason, the system does not discard the lead. It saves the raw message with a generic response and sets the warning field in the API response. This ensures no lead is silently lost during an outage.

Temperature of 0.2 — A low temperature was chosen deliberately. Lead classification and field extraction are not tasks that benefit from creativity. A low temperature makes the model's output stable and repeatable, which is essential for a system that feeds into a database.

4. Edge Cases Handled

Empty message — Caught at the validation layer before any database or LLM interaction. Returns HTTP 400 with a descriptive error message.

Message exceeding 5,000 characters — Rejected at input validation. Prevents excessively long inputs from inflating API token usage or causing unexpected model behavior.

LLM API timeout — The HTTP client has a 30-second timeout. On timeout, the request is retried up to three times with exponential backoff (2 seconds, then 4 seconds, then 6 seconds) before the fallback is activated.
HTTP 429 rate limit from Groq — Treated the same as any other HTTP error. The retry logic handles it with backoff, giving the rate limit window time to reset before the next attempt.
Malformed JSON from the LLM — Even with a well-structured prompt, models can occasionally return text that is not valid JSON. This is caught by a JSONDecodeError handler and the fallback response is used instead of crashing.
Intent value outside allowed set — After parsing the JSON, the intent field is checked against the three allowed values. Any unexpected value is corrected to Support, which is the safest default for an unknown lead type.
No API key configured — The system checks for the API key at the start of the LLM handler. If it is missing or still set to the placeholder value, the fallback is activated immediately without making any outbound network request.
Missing frontend file — If index.html is not found at the expected path, the root route returns a descriptive HTML error page showing exactly where the file was expected, rather than a generic 500 error.
Lead always persisted — In every code path, including all error and fallback paths, the lead is written to the database before the response is returned. There is no code path where a received message is discarded silently.

--- END OF ASSESSMENT 1 ---

Assessment 2

Mysoft Heaven (BD) Ltd. Specific AI Chatbot (RAG) with Web Interface
Project Title: Company-Specific RAG Chatbot for Mysoft Heaven (BD) Ltd. Technology Stack: Python, Flask, FAISS, Sentence-Transformers, Groq API (LLaMA 3.3), HTML/CSS/JavaScript

1. Project Overview

This project implements a Retrieval-Augmented Generation (RAG) chatbot designed exclusively for Mysoft Heaven (BD) Ltd. The chatbot answers questions strictly based on company-provided documents and information. It does not generate answers from general world knowledge, and it explicitly rejects questions that fall outside the company's knowledge base.
The system is composed of three layers: a knowledge ingestion pipeline that processes and indexes company documents, a backend API server that handles retrieval and language model inference, and a web-based frontend interface through which users interact with the chatbot in real time.

2. Application Flow

When a user types a question into the web interface and submits it, the following sequence of operations takes place.
Step 1 - The frontend (index.html) captures the user's message and sends an HTTP POST request to the Flask backend at the /chat endpoint. The request body includes the message text and a session identifier that is unique to the browser tab.
Step 2 - The Flask server (app.py) receives the request, validates that the message field is present and non-empty, and passes the message along with the session identifier to the RAGChatbot class in rag_engine.py.
Step 3 - The RAGChatbot first encodes the user's question into a 384-dimensional vector using the local sentence-transformers model (all-MiniLM-L6-v2). This embedding represents the semantic meaning of the question as a point in vector space.
Step 4 - The embedding is passed to the FAISS vector index, which performs a nearest-neighbour search across all pre-indexed document chunks. The search returns the four most similar chunks together with their cosine similarity scores.
Step 5 - The similarity score of the best-matching chunk is compared against a threshold of 0.35. If the score falls below this threshold, the system concludes that no relevant company information exists for this query and returns a fallback response immediately, without contacting the language model.
Step 6 - If the score meets the threshold, the retrieved chunks are assembled into a structured context block. This context, combined with the conversation history for the current session and a strict system prompt, is sent to the Groq API.
Step 7 - The Groq API runs inference on the LLaMA 3.3 70B model and returns a natural language answer grounded in the provided context.

Step 8 - The backend packages the answer, the numerical confidence score, a human-readable confidence label, and the names of the source sections that contributed to the answer into a JSON response.
Step 9 - The frontend receives the response and renders the answer in the chat window, displaying the confidence badge and source section tags beneath each bot message.

3. AI Logic and Automation

3.1 Document Chunking

The company knowledge base, defined in company_data.py, is divided into ten thematic sections covering the company overview, services, products, technology stack, certifications, project portfolio, team, contact information, mission and vision, and support policy.

Each section is split into overlapping character-level chunks of 500 characters with a 100-character overlap between consecutive chunks. The overlap ensures that a sentence that happens to fall at the boundary between two chunks is fully represented in at least one of them, preventing information loss. Every chunk carries metadata identifying its source section and company, which allows the system to report back to the user precisely which part of the knowledge base was used to construct the answer. On first startup, these chunks are embedded and saved to disk as a FAISS index. On all subsequent startups, the saved index is loaded directly, so there is no re-embedding cost.

3.2 Embedding Model

The embedding model used is sentence-transformers/all-MiniLM-L6-v2. This model was chosen for the following reasons. It runs entirely on the local CPU with no API key or internet connection required at inference time, keeping operational costs at zero. It produces 384-dimensional vectors, which are compact enough for fast FAISS search while being expressive enough to capture the semantic meaning of short-to-medium length factual queries. The model downloads once (approximately 90 MB) and is cached locally for all subsequent runs.

All vectors are L2-normalised before being stored in or queried against the FAISS index. This means the inner-product search performed by FAISS (IndexFlatIP) is mathematically equivalent to cosine similarity, which measures directional similarity between vectors rather than their absolute magnitude.

3.3 Retrieval and Ranking

On each query, the top four most similar chunks are retrieved from the FAISS index. These four chunks are concatenated with their section labels and passed as the context window to the language model. Using four chunks rather than one provides the model with richer context while keeping the prompt within a manageable token count.

3.4 Language Model and Prompt Design

The language model used for generation is LLaMA 3.3 70B Versatile, accessed through the Groq API on the free tier. The system prompt passed to the model on every request instructs it explicitly to answer only from the provided context, to avoid fabricating facts, prices, or names, and to redirect the user to the company contact email if the answer cannot be found in the context. The temperature parameter is set to 0.2, which produces more deterministic and factually conservative responses, reducing the risk of the model introducing information not present in the retrieved chunks.

3.5 Conversation Memory

Each user session is assigned a unique session identifier generated in the browser at page load time. The backend maintains a dictionary mapping session identifier to message histories. On each request, the last six message pairs (twelve messages) from the current session are prepended to the LLM prompt, enabling the model to answer follow-up questions that refer to earlier parts of the conversation. The history is capped at twelve messages to keep prompt length bounded. When the user clicks the Clear Chat button, the frontend sends a POST request to the /clear endpoint, which removes the session's history from the dictionary.

3.6 Confidence Scoring

The cosine similarity score returned by FAISS for the best-matching chunk serves as the basis for the confidence score displayed to the user. This score is multiplied by 100 and rounded to one decimal place to produce a percentage. The percentage is then mapped to a human-readable label as follows: a score of 60 percent or above is labelled High Confidence, a score between 45 and 59 percent is labelled Medium Confidence, a score between 35 and 44 percent is labelled Low Confidence, and anything below 35 percent is labelled Out of Scope. These labels are displayed alongside the answer in the frontend as coloured badges, giving the user a clear indication of how confidently the system matched their question to the available information.

4. Key Design Decisions

4.1 Why FAISS Over a Cloud Vector Database

FAISS (Facebook AI Similarity Search) was selected instead of a cloud-based vector database such as Pinecone or Weaviate because the project operates on a free budget and the knowledge base is small and static. FAISS stores the index as two local files (index.faiss and chunks.json) and loads them in under a second on startup. For a company knowledge base of this scale, a cloud database would introduce unnecessary latency, cost, and external dependency with no performance benefit.

4.2 Why Groq Instead of OpenAI

The Groq API was chosen because it offers a free tier with no credit card requirement and provides inference on the LLaMA 3.3 70B model at speeds significantly faster than comparable OpenAI models. For this assessment, the free rate limit of approximately 14,400 requests per day is more than sufficient.

4.3 Strict Context Grounding

The most important design constraint in this project is that the chatbot must never answer from general knowledge. This is enforced at two levels. The first level is the similarity threshold check, which prevents the language model from being invoked at all if the query does not match any company content with sufficient confidence. The second level is the system prompt, which tells the model that its only permitted knowledge source is the context block it has been given. Together, these two mechanisms ensure that the chatbot behaves as a document-grounded assistant rather than a general-purpose chatbot.

4.4 Stateless API with Stateful Memory

The Flask API is stateless in the HTTP sense — each request is self-contained — but the RAGChatbot class maintains in-memory conversation histories keyed by session identifier. This design keeps the API simple and testable while still supporting multi-turn dialogue. The trade-off is that conversation history is lost if the server restarts, which is acceptable for this assessment context.

4.5 Single-File Frontend

The entire web interface is contained in a single HTML file with no build tools, no bundler, and no external framework. This makes deployment and portability trivial: the file can be opened directly in any browser without a web server.

5. Handled Edge Cases

5.1 Out-of-Scope Queries

If a user asks a question that has no relation to Mysoft Heaven's information, such as general knowledge questions, questions about competitors, or unrelated personal queries, the cosine similarity score of the best matching chunk will fall below 0.35. In this case the language model is not called. The user receives a clear, polite message explaining that the question is outside the chatbot's knowledge base and is directed to the company's contact email.

5.2 Empty or Whitespace-Only Messages

The backend validates the incoming request body and strips whitespace from the message field before processing. If the message is empty after stripping, the server returns a 400 Bad Request response with a descriptive error message. The frontend also prevents submission if the input field is empty.

5.3 Missing or Malformed Request Body

If the POST /chat request arrives without a JSON body or without the required message field, the server returns a 400 Bad Request response. This prevents unhandled exceptions from propagating to the client.

5.4 Unhandled Server Exceptions

The /chat endpoint wraps the entire processing logic in a try-except block. If any unexpected exception occurs at any stage of the pipeline, the full stack trace is printed to the server console for debugging, and the client receives a 500 Internal Server Error response with a generic message. This prevents the server from exposing internal implementation details to the end user.

5.5 FAISS Index Rebuild on Missing Index

When the server starts, it first attempts to load an existing FAISS index from disk. If the index files do not exist, it automatically runs the full document chunking and embedding pipeline and saves the resulting index to disk before accepting any requests. This means the system is self-initialising and requires no manual setup step beyond providing the API key.

5.6 Conversation History Overflow

The per-session conversation history is bounded to a maximum of twelve messages (six turns). When this limit is exceeded, the oldest messages are dropped from the front of the list. This prevents the prompt from growing indefinitely, which would eventually exceed the language model's context window and cause API errors.

5.7 Confidence Transparency

Rather than silently returning answers of varying reliability, the system always surfaces the confidence score and source sections to the user. When confidence is low, the user can see this in the interface and exercise appropriate judgment about the answer's reliability, rather than trusting a response that may have been produced from a weak retrieval match.

6. Multi-Company Architecture

The system is designed to support multiple companies without requiring changes to the core pipeline. Each company is assigned a unique company_id string. The FAISSVectorStore class stores its index under vector_store/{company_id}/, so the indexes for different companies are fully isolated from one another on disk. The system prompt and fallback messages are parameterised with the company's name and contact details, which are defined in the company's document file. To onboard a new company, it is sufficient to create a document file for that company, build a new FAISS index for it, and route incoming requests carrying that company's identifier to the corresponding index. The retrieval logic, embedding model, and language model integration require no modification.

7. Technology Summary

Component Technology Used Language Model LLaMA 3.3 70B Versatile via Groq API (free tier) Embedding Model sentence-transformers/all-MiniLM-L6-v2 (local) Vector Database FAISS (faiss-cpu, local file-based) Backend Framework Flask with Flask-CORS Frontend HTML5, CSS3, Vanilla JavaScript (single file) Environment Config python-dotenv Language Python 3.10+

--- END OF ASSESSMENT 2 ---

Assessment 3

SocialAI — Application Documentation

1. Application Flow

The application follows a linear pipeline that begins when a company logs into the dashboard and ends with a qualified lead being synced to the CRM system.

When a user first visits the platform, they are presented with a login screen. Authentication is handled using JWT tokens. Upon successful login, the server issues a signed token that the frontend stores in localStorage and attaches to every subsequent API request. This ensures that each company operates in complete isolation — no user can access another company's data because every database query is filtered by the user ID extracted from the token.

After login, the user lands on the Dashboard. From here, they can simulate or receive an incoming social media message. The message enters a three-stage AI processing pipeline. First, the message is analyzed for sentiment, intent, and urgency. Second, an automated reply is generated based on the message content and the platform it came from. Third, the sender is classified as a Hot, Warm, or Cold lead based on the analysis. All three results are saved to the database simultaneously and returned to the frontend, where the user sees the AI reply, sentiment label, and lead tag in real time.

The Leads page displays all classified contacts. Users can filter by tag and manually override any AI-assigned classification if the business context requires it. The CRM page allows users to push all Hot and Warm leads into the CRM database with a single click. Only qualified leads are synced — Cold leads are excluded by design to keep the CRM clean.

2. AI Logic and Automation

The AI layer consists of three independent agents, all powered by Groq's LLaMA3-8B-8192 model. Each agent is given a tightly scoped system prompt that constrains its output to a specific task.

Agent 1 — Message Analyzer

This agent receives the raw message text and is instructed to return a structured JSON object containing four fields: sentiment (positive, negative, or neutral), intent (buying, inquiry, complaint, compliment, or other), urgency (high, medium, or low), and a list of keywords. Using a structured output format rather than free text makes the downstream agents more reliable because they receive consistent, parseable data.

Agent 2 — Reply Generator

This agent receives both the original message and the platform name. The system prompt instructs it to respond as a professional social media manager, keep the reply to two or three sentences, and never make up specific prices or promises. The platform name is passed deliberately because the appropriate tone differs — a LinkedIn reply should be more formal than an Instagram reply.

Agent 3 — Lead Classifier

This agent receives the original message and the full output of Agent 1. It is instructed to return a single word: hot, warm, or cold. The definitions provided in the prompt are precise. Hot means the person is ready to buy, asking for pricing, or showing high urgency. Warm means they are interested but not ready to commit. Cold means they are browsing, complaining without purchase intent, or sending an unrelated message. By feeding Agent 1's output into Agent 3, the classifier benefits from structured context rather than having to re-derive intent from raw text.

Automation Logic

The entire three-agent sequence runs automatically every time a message arrives. No human action is required. If any agent call fails, the system does not crash — it falls through to a rule-based fallback that uses keyword matching to assign sentiment and lead tags. This means the automation continues functioning even during API outages, which is critical for a production SaaS product where downtime directly affects customer experience.

3. Key Design Decisions

Why Groq instead of OpenAI

Groq provides a free tier with fast inference speeds using open-source LLaMA3 models. For a student assessment project, this removes the cost barrier entirely while still demonstrating real AI integration. The code is structured so that switching to a different provider requires changing only the client initialization in groq_agent.py — the rest of the system is provider-agnostic.

Why three separate agents instead of one

A single prompt asking for analysis, a reply, and a lead tag simultaneously produces unreliable results because the model has to balance three different output formats at once. Separating the tasks into three focused agents produces cleaner outputs, makes each agent independently testable, and allows the system to swap or upgrade individual agents without affecting the others. This follows the single responsibility principle applied to AI design.

Why SQLite for development

SQLite requires no server setup, runs as a file, and makes the project immediately runnable on any machine. The database layer is abstracted through SQLAlchemy, so migrating to PostgreSQL for production requires changing a single connection string. This makes local development frictionless while keeping the production path clear.

Why JWT over session-based authentication

JWT is stateless, meaning the server does not need to store session data. This makes the backend easier to scale horizontally — any server instance can validate a token without querying a session store. It also allows the React frontend and the Flask backend to run on different ports during development without sharing cookies.

Why the frontend uses a proxy

The package.json includes a proxy setting that forwards all /api/* requests from port 3000 to port 5000. This avoids CORS issues during development and means the frontend code never needs to hardcode a backend URL — it simply calls /api/messages and the proxy handles routing.

Why leads are not auto-deleted

Once a Cold lead is created, it is never automatically removed. Instead, users can manually change the tag. This is intentional because a Cold lead today may become a Hot lead after a follow-up campaign. Preserving the history gives businesses a complete record of all interactions.

4. Handled Edge Cases

Groq API unavailability

If the Groq API key is missing or the API is down, every agent function catches the exception and falls back to rule-based logic. The fallback uses keyword matching to determine sentiment and intent, then applies scoring rules to assign a lead tag. The message is still saved, the reply is still generated, and the lead is still tagged — the only difference is reduced accuracy compared to the LLM output.

Malformed AI responses

The LLaMA3 model occasionally wraps its JSON output in markdown code fences or adds explanatory text before the JSON object. Agent 1 handles this by scanning the response string for the first { and last } characters and extracting only that substring before parsing. If parsing still fails, the fallback analysis is used instead.

Duplicate leads

If the same sender name submits multiple messages, the system does not create duplicate lead records. It checks for an existing lead with the same name under the same user account, and if found, updates the tag, increments the message count, and refreshes the last interaction timestamp. This keeps the lead list clean and the CRM sync accurate.

CRM sync idempotency

Running the CRM sync multiple times does not create duplicate contacts. The sync endpoint checks whether a contact with the same lead ID already exists in the CRM table. If it does, it updates the lead score and message count rather than inserting a new row. This means users can safely click Sync Now as many times as they want without corrupting their CRM data.

Expired or invalid JWT tokens

If a user's token has expired or been tampered with, Flask-JWT-Extended rejects the request with a 401 response. The React frontend's Axios instance receives this response and the user is redirected to the login page automatically. The token expiry is set to 24 hours, which balances security with usability for a business dashboard used daily.

Empty message simulation

If a user submits the simulate form with an empty message, the AI agents still run but the fallback logic assigns a neutral sentiment, an inquiry intent, and a cold tag. The system does not throw an error — it processes the empty input gracefully and saves a record. This prevents the pipeline from breaking under unexpected input.

Social platform disconnection

If a social platform is disconnected while messages are being processed, the platform field on the message record retains the original value. The system does not fail or orphan records — it simply marks the reply as sent and continues processing. This ensures data consistency even when external connections are interrupted.

Assessment Summary

This project demonstrates end-to-end system thinking across six dimensions required by the assessment.

The AI agents and services layer uses three specialized LLaMA3 agents for analysis, reply generation, and lead classification, with a complete fallback system for resilience. The frontend to backend to AI data flow is linear and traceable — React sends a request, Flask processes it, Groq generates the intelligence, and the result flows back to the UI in a single round trip. Authentication and authorization use industry-standard JWT with bcrypt password hashing and per-query data isolation. Data security is enforced through environment-based secret management, CORS restrictions, and never storing sensitive values in code. Cost optimization is achieved by using Groq's free tier, separating agents to avoid redundant calls, and implementing fallback logic that avoids API calls entirely for simple cases. Failure scenarios including API outages, malformed responses, duplicate records, and expired tokens are all handled explicitly with defined recovery behavior rather than left to produce unhandled exceptions.

--- END OF ASSESSMENT 3 ---